



Music Ontology

Group F Report

VU Semi-Automatic Information and Knowledge Systems

Submitted by:

Muhammad Umer Raza	12436441
Margarida M A Gonçalves	12142151

June 25, 2026

1 Introduction

Music ontology is a classical example used in Semantic Systems. It is a complex domain, with different entities, such as artists, concerts and record labels, and multiple relationships which are inherently intricate, multi-directional and difficult to capture in a traditional relational schema without significant loss of semantic expressiveness.

This project applies the full semantic web technology stack to the music domain, constructing a formal knowledge graph that connects artists, albums, tracks, concerts, tours, venues, genres, awards, record labels, instruments, composers and producers under a shared ontological vocabulary. The ontology was created with Protégé, incorporating three Ontology Design Patterns and advanced OWL 2 features like property chains or disjoint classes. The next step involved creating a relational database populated with real Spotify track and artist data augmented with some synthetic data, which was then materialized into a knowledge graph of 73,243 RDF triples via the Ontop OBDA framework. Step 3 verified the ontology using the OOPS! pitfall scanner and validated the knowledge graph with SHACL shapes of varying complexity, followed by the ontology alignment with the Performed Music Ontology and BIBFRAME using AgreementMakerLight. Lastly, we developed a Retrieval-Augmented Generation (RAG) interface so that an LLM can answer complex questions grounded in structured, verified knowledge-graph data.

The result is a semantically rich, queryable representation of the popular music landscape that goes beyond what a relational database can express.

2 Step 1: Advanced Ontology Engineering

For this project, we decided to reuse the ontology already used in the course Introduction to Semantic System (ISS), but adapt and extend, since the original ontology focused on classical music and we wanted to shift it for popular music and concerts.

Hence, our domain is the comprehensive ecosystem for music and events such as concerts with the goal of connecting creative intellectual property (albums, tracks, genres) with physical event contexts and support semantic search and reasoning.

In order to guide the creation of the onlogy we decided on the following competency questions related with Artists, Albums and Events:

1. Which artists perform at a given concert?
2. Which tracks belong to a given album and what genres do they cover?
3. What is the relationship between a venue's capacity and the actual concert audience?

During ontology engineering, we used some advanced modeling features offered by OWL2 and 3 different Ontology Design Patterns (ODPs). Some OWL2 features used include:

- **Disjoint classes:** Band $\sqcap$ SoloArtist are disjoint
- **Cardinality restrictions:** Album has at least 1 Track (Existential Restriction)
- **Equivalent classes:** LiveAlbum $\equiv$ Album $\sqcap$ (recordedAt some Concert)
- **Comments and Meta Data:** Rdfs:label & rdfs:comment
- **Property chains:** hasMember o playsInstrument $\rightarrow$ featuresInstrument
- **Symmetric Properties:** Applied to collaboratedWith (Musician $\rightarrow$ Musician). If Artist A is asserted to have collaborated with Artist B, the inverse inference is materialized automatically, preventing data duplication in the KG layer.

Regarding distinct Ontology Design Patterns (ODPs) we used *Part-of*, which reifies a complex part-whole architecture mapping a Concert Program to a structured sequence of Program Items (individual track performances), *Participation Pattern*, which reifies the artist-concert relationship so multiple artists can hold

different roles at the same event (Class created to deal with N-ary relationship), and *Typed Situation Pattern*, which was used as class too, in order to capture the whole context of a situation, for example an Award.

The resulting ontology is made up of 56 classes grouped into Person, Venue, Record Label, Music Work, Instrument, Genre, Event and Award. These are our main classes, which then have subclasses and shown in Table 1.

Table 1: Ontology Class and Subclass Hierarchy

Parent Class	Subclass / Hierarchy Level
Award	
Event	↪ Concert ↪ MusicFestival ↪ RecordingSession ↪ Tour
Genre	↪ ElectronicGenre ↪ LatinGenre ↪ PopGenre ↪ RapGenre ↪ RnBGenre ↪ RockGenre
Instrument	↪ BrassInstrument→ Trumpet ↪ KeyboardsInstrument→ Piano ↪ PercussionInstrument→ Drums ↪ StringInstrument→ Guitar, Violin ↪ WoodwindInstrument→ Flute
MusicWork	↪ Album ↪ EP ↪ LiveAlbum ↪ StudioAlbum ↪ Program ↪ ProgramItem ↪ Track
Participation	↪ CompositionParticipation ↪ PerformanceParticipation ↪ ProductionParticipation
Person	↪ Composer ↪ Musician ↪ Band ↪ SoloArtist ↪ Producer
RecordLabel	
Situation	↪ AwardSituation ↪ CollaborationSituation ↪ MembershipSituation
Venue	↪ Arena ↪ ConcertHall ↪ OutdoorVenue ↪ Stadium ↪ Studio

Table 2: Ontology Data Properties Specification

Data Property	Range	Description / Domain
mo:name	xsd:string	General name identifier for several entities (Genre, Label, Venue, Tour, etc.)
mo:title	xsd:string	Title for events or music works (Track, Album, Concert)
mo:releaseYear	xsd:gYear	Calendar year when an Album was released
mo:eventDate	xsd:date	Calendar date of an Event (Concert, Tour, RecordingSession)
mo:eventTime	xsd:time	Timestamp of when an Event starts
mo:ticketPrice	xsd:decimal	Price in euros of a ticket for a Concert
mo:audienceCount	xsd:integer	Number of people attending an Event
mo:durationInSeconds	xsd:positiveInteger	Exact duration in seconds of a Track (functional)
mo:capacity	xsd:positiveInteger	Maximum number of people possible at a Venue (functional)
mo:location	xsd:string	Geographical location of a Venue or RecordLabel
mo:description	xsd:string	Textual description; no domain restriction (used broadly)
mo:entityCode	xsd:integer	Numeric business key used as unique class identifier (functional)
mo:instrumentType	xsd:string	Classification of the category of an Instrument
mo:firstName	xsd:string	Given name of a Person
mo:lastName	xsd:string	Legal surname of a Person
mo:nationality	xsd:string	Nationality of a Person
mo:biography	xsd:string	Short biographical summary of a Person (artist)
mo:birthDate	xsd:date	Calendar date of birth of a Person
mo:id	xsd:integer	Functional primary key identifier for a Musician
mo:position	xsd:positiveInteger	Sorting index of a Track within a MusicWork (functional)
mo:rating	xsd:integer	Rating of a MusicWork on a scale from 0 to 10
mo:trackCount	xsd:positiveInteger	Number of tracks in an Album or playlist

Table 3: Ontology Object Properties Specification

Object Property	Domain $\rightarrow$ Range	Description / Notes
mo:hasCreator	MusicWork $\rightarrow$ Person	Identifies the author or composer responsible for a MusicWork
mo:createdWork	Person $\rightarrow$ MusicWork	Inverse of hasCreator ; links a Person to works they created

continued on next page

Table 3 – *continued from previous page*

Object Property	Domain → Range	Description / Notes
mo:hasGenre	MusicWork → Genre	Classifies a MusicWork under a specific stylistic Genre
mo:isGenreOf	Genre → MusicWork	Inverse of hasGenre
mo:hasTrack	- → Track	Relates an Album or playlist to its individual Tracks
mo:trackIsPartOf	Track → -	Inverse of hasTrack; locates a Track within its parent Album
mo:hasMember	- → Musician	Links a Band or Orchestra to its constituent individual Musicians
mo:isMemberOf	Musician → -	Connects a Musician to their Band or Orchestra
mo:playsInstrument	Musician → Instrument	Links a Musician to the Instruments they play
mo:isPlayedBy	Instrument → Musician	Inverse of playsInstrument
mo:featuresInstrument	- → Instrument	Infers instruments featured in a group via chain hasMember ◦ playsInstrument
mo:isFeaturedIn	Instrument → -	Inverse of featuresInstrument
mo:hostedAt	Event → Venue	Binds a scheduled Event to its physical Venue (functional)
mo:hostedEvent	Venue → Event	Inverse of hostedAt (inverse functional)
mo:recordedAt	Album Track → Venue	Identifies the Venue where an Album or Track was recorded; chain participatesIn ◦ hostedAt
mo:isRecordingVenueFor	Venue → Album Track	Inverse of recordedAt
mo:hasParticipant	Event → Person	Connects an Event to a Person involved in it
mo:participatesIn	Person → Event	Inverse of hasParticipant
mo:hasParticipation	Person → Participation	Links a Person to their reified Participation ODP wrapper
mo:isParticipationOf	Participation → Person	Inverse of hasParticipation
mo:isPartOfTour	Concert MusicFestival → Tour	Links a Concert or Festival to the Tour it belongs to
mo:occursDuring	Tour → Concert MusicFestival	Inverse of isPartOfTour
mo:producedBy	MusicWork → Producer	Links a MusicWork to its Producer
mo:isProducerOf	Producer → MusicWork	Inverse of producedBy
mo:hasReleased	RecordLabel → MusicWork	Indicates a RecordLabel published a MusicWork
mo:releasedUnder	MusicWork → RecordLabel	Inverse of hasReleased
mo:awardGivenTo	Award → Person	Relates an Award to the Person who received it
mo:receivedAward	Person → Award	Inverse of awardGivenTo
mo:hasProgramItem	Program → ProgramItem	Relates a concert Program to its individual ProgramItem entries

continued on next page

Table 3 – continued from previous page

Object Property	Domain → Range	Description / Notes
mo:isProgramItemOf	ProgramItem → Program	Inverse of hasProgramItem
mo:collaboratedWith	Musician → Musician	Symmetric and reflexive mutual creative collaboration between Musicians
mo:involvesPerson	Situation → Person	Declares a Person as a key participant in a reified Situation (ODP)
mo:isInvolvedInSituation	Person → Situation	Inverse of involvesPerson
mo:involvesMusicWork	Situation → MusicWork	Declares a MusicWork as an element inside a reified Situation
mo:isMusicWorkInvolved	MusicWork → Situation	Inverse of involvesMusicWork
mo:hasSetting	Thing → Situation	Links contextual entities to their governing Situation wrapper
mo:isSettingFor	Situation → Thing	Inverse of hasSetting

3 Step 2: KG creation through OBDA

After creating the Ontology we tried to find data to populate it. We found a dataset in kaggle [1], which has data regarding songs on Spotify, but it still lacked data for the artists and concerts. We decided to use a subset of these dataset, since it had a lot of duplicated songs. For rest of the data we just generated it so we would have more materialized classes.

The resulting tables of our database are in Table 4

Table 4: Database Schema Overview

Table	Columns	Primary Key / Notes
ARTIST	ARTIST_ID, ARTIST_NAME, MUSICIAN_TYPE, LABEL_ID	PK: ARTIST_ID; FK: LABEL_ID
ALBUM	ALBUM_ID, ALBUM_NAME, ARTIST_ID, RELEASE_DATE	PK: ALBUM_ID; FK: ARTIST_ID
ALBUMTYPE	ALBUM_TYPE_ID, ALBUM_ID, ALBUM_TYPE	PK: ALBUM_TYPE_ID; FK: ALBUM_ID; values: StudioAlbum, LiveAlbum, EP
TRACK	TRACK_ID, TRACK_NAME, ALBUM_ID, GENRE_ID, DURATION_MS, POPULARITY	PK: TRACK_ID; FK: ALBUM_ID, GENRE_ID
GENRE	GENRE_ID, GENRE_NAME	PK: GENRE_ID; 6 genres (pop, rap, rock, latin, r&b, edm)
VENUE	VENUE_ID, NAME, VENUE_TYPE, LOCATION, CAPACITY	PK: VENUE_ID; types: Arena, Theater, ConcertHall, Studio, OutdoorVenue
CONCERT	CONCERT_ID, TITLE, EVENT_DATE, EVENT_TIME, TICKET_PRICE, AUDIENCE, VENUE_ID, ARTIST_ID, TOUR_ID	PK: CONCERT_ID; FK: VENUE_ID, ARTIST_ID, TOUR_ID
TOUR	TOUR_ID, NAME, START_DATE, END_DATE, ARTIST_ID	PK: TOUR_ID; FK: ARTIST_ID
RECORDLABEL	LABEL_ID, NAME, LOCATION	PK: LABEL_ID

continued on next page

Table 4 – continued from previous page

Table	Columns	Primary Key / Notes
AWARD	AWARD_ID, NAME, YEAR, CATEGORY, ARTIST_ID	PK: AWARD_ID; FK: ARTIST_ID
INSTRUMENT	INSTRUMENT_ID, NAME, INSTRUMENT_TYPE, OWL_CLASS	PK: INSTRUMENT_ID; maps to OWL class hierarchy
ARTISTINSTRUMENT	ARTIST_ID, INSTRUMENT_ID	PK: (ARTIST_ID, INSTRUMENT_ID); junction table
PRODUCER	PRODUCER_ID, FIRST_NAME, LAST_NAME, NATIONALITY	PK: PRODUCER_ID
COMPOSER	COMPOSER_ID, FIRST_NAME, LAST_NAME, NATIONALITY, BIRTH_DATE	PK: COMPOSER_ID
ALBUMPRODUCER	ALBUM_ID, PRODUCER_ID	PK: (ALBUM_ID, PRODUCER_ID); junction table
ALBUMCOMPOSER	ALBUM_ID, COMPOSER_ID	PK: (ALBUM_ID, COMPOSER_ID); junction table
RECORDINGSESSION	SESSION_ID, SESSION_DATE, ALBUM_ID, VENUE_ID	PK: SESSION_ID; FK: ALBUM_ID, VENUE_ID
MUSICFESTIVAL	FESTIVAL_ID, FESTIVAL_NAME, VENUE_ID, START_DATE	PK: FESTIVAL_ID; FK: VENUE_ID
CONCERTTRACK	CONCERT_ID, TRACK_ID, POSITION	PK: (CONCERT_ID, TRACK_ID); junction table with ordering

We uploaded the data to H2 and created R2RML mappings to link our ontology to the database. Afterwards we materialized the triples and got the resulting file `music_ontology-materialized.ttl`.

We imported it into GraphDB and got 93,045 total statements, 73,243 explicit and 19,802 inferred. The GraphDB *Class Hierarchy* explorer (see screenshot, Figure 1) shows 51 data classes in a bubble chart. MusicWork and Track appear as the two largest bubbles, reflecting the dataset’s track-centric nature.

Class hierarchy ⓘ



Class Count ⓘ

51

1

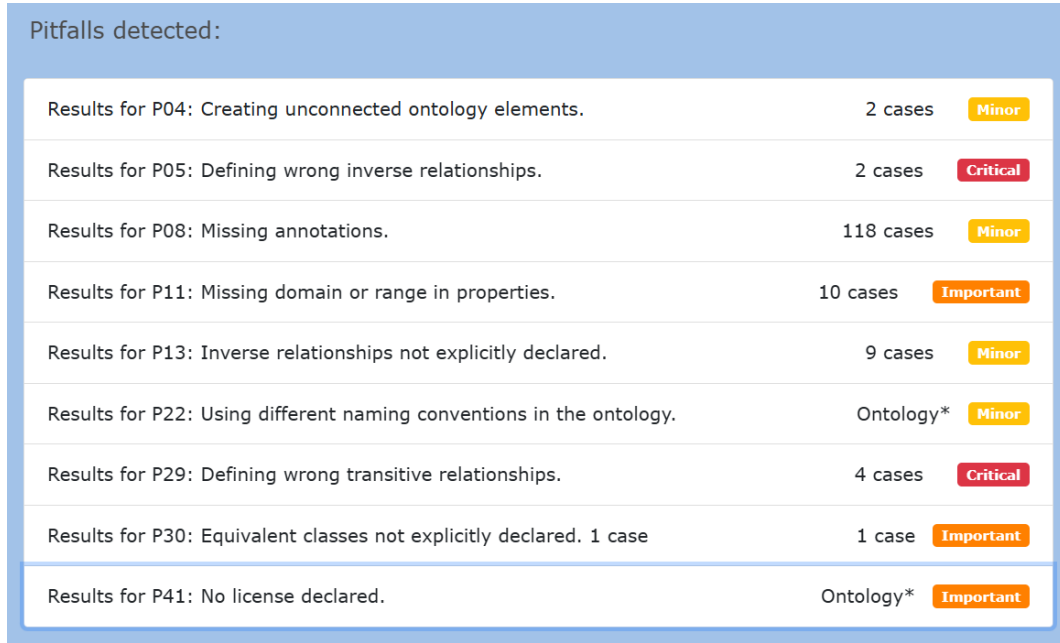


Figure 1: GraphDB Class Hierarchy explorer for the music_kg repository

4 Step 3: Verification of Semantic Structures

4.1 OOPS

In step 3, we submitted the ontology to the OOPS! Ontology Pitfall Scanner and got the results are shown in 2



Pitfalls detected:		
Results for P04: Creating unconnected ontology elements.	2 cases	Minor
Results for P05: Defining wrong inverse relationships.	2 cases	Critical
Results for P08: Missing annotations.	118 cases	Minor
Results for P11: Missing domain or range in properties.	10 cases	Important
Results for P13: Inverse relationships not explicitly declared.	9 cases	Minor
Results for P22: Using different naming conventions in the ontology.	Ontology*	Minor
Results for P29: Defining wrong transitive relationships.	4 cases	Critical
Results for P30: Equivalent classes not explicitly declared. 1 case	1 case	Important
Results for P41: No license declared.	Ontology*	Important

Figure 2: OOPS! evaluation results

Table 5: OOPS! Pitfalls Detected – Full List with Resolutions

Code	Description	Severity	Resolution / Remark
P04	Creating unconnected ontology elements	Minor	These classes were leftover from the first ontology, so we just deleted them
P05	Defining wrong inverse relationships	Critical	Reviewed and corrected inverse property declarations to ensure logical symmetry
P08	Missing annotations	Minor	Added <code>rdfs:label</code> and <code>rdfs:comment</code> to all classes and properties
P11	Missing domain or range in properties	Important	Explicitly declared domain and range for all object and data properties
P13	Inverse relationships not explicitly declared	Minor	Added explicit <code>owl:inverseOf</code> declarations for all paired properties
P22	Using different naming conventions	Minor	Standardised all class and property names
P29	Defining wrong transitive relationships	Critical	Reviewed the transitive properties for logical correctness
P30	Equivalent classes not explicitly declared	Important	Left it
P41	No license declared	Important	Left it

Most of the pitfalls happened because of distraction or forgetfulness, so they were quickly fixed.

4.2 SHACL

After checking oops, we used SHACL rules to check out materialized KG. We tried first with SHACL4Protege plugin from Protégé, but our validation results were not showing any violation even with wrong data introduced, hence we had to switch to TopBraid SHACL engine.

Ten SHACL shapes of varying complexity were defined across two files (music_shapes.ttl and concert_shape.ttl) and validated against the knowledge graph. The shapes cover structural, cardinality, datatype, value-range, and SPARQL-constraint rules.

Table 6: SHACL Shapes Specification and Complexity

Shape	Description	Target	Complexity
ArtistShape	Musician must have a name	Musician	Simple
VenueShape	Venue must have a name and integer capacity	Venue	Simple
ConcertShape	Concert must have title, date and price	Concert	Simple
AlbumShape	Album must have a title	Album	Simple
TrackShape	Track must have a title	Track	Simple
GenreShape	Genre must have exactly one name	Genre	Medium
TourShape	Tour must have a name	Tour	Simple
RecordLabelShape	RecordLabel must have a name	RecordLabel	Simple
ConcertAudienceShape	Concert audience count must be positive integer	Concert	Medium
NoSelfGenreShape	Track must not be its own genre	Track	SPARQL

The first Validation Report showed some typos in the mappings, for example in the venue mapping there was a typo in the url: `<http://example.org/music-kg/music-kg/venue/17>` instead of `<http://example.org/music-kg/venue/17>`

```

1 sh:result
2   [rdf:type                sh:ValidationResult;
3     sh:focusNode           <http://example.org/music-kg/music-kg/venue/17>;
4     sh:resultMessage       "Venue must have a name.";
5     sh:resultPath          :name;
6     sh:resultSeverity      sh:Violation;
7     sh:sourceConstraintComponent sh:MinCountConstraintComponent;
8     sh:sourceShape         _:b5];

```

Listing 1: Validation Result Example (from music_shapes.ttl)

The other problem we found was with the reasoning. By defining wrong the Domain of the data property **title** in our ontology, the reasoner was wrongly inferring that an **Award** was a **Concert** as well, like we can see in:

```

1 sh:result
2   [ rdf:type                sh:ValidationResult;
3     sh:focusNode           <http://example.org/music-kg/award/13>;
4     sh:resultMessage       "Concert must have a ticket price.";
5     sh:resultPath          :ticketPrice;
6     sh:resultSeverity      sh:Violation;
7     sh:sourceConstraintComponent sh:MinCountConstraintComponent;
8     sh:sourceShape         _:b4 ];

```

Listing 2: Validation Result Example (from music_shapes.ttl)

After correcting these, we were able to run the rules and get a positive validation result.

5 Step 4: Ontology Alignment

5.1 Target Ontology

The music domain ontology (<http://example.org/music-ontology>) was aligned with the **Performed Music Ontology (PMO)** (<http://performedmusicontology.org/ontology/>), a well-established ontology for representing performed music, musical events, recordings, and performance-related concepts. PMO is built upon Semantic Web standards and reuses concepts from BIBFRAME and related cultural heritage ontologies.

5.2 Alignment Tool

The alignment was performed using **AgreementMakerLight (AML)**, an ontology matching system that combines lexical similarity, structural matching, and background knowledge sources to identify semantic correspondences between ontology entities.

AML automatically compares classes, properties, and ontology structures to discover equivalent concepts and produces mappings in the standard Alignment API RDF format.

5.3 Alignment Result

The alignment is serialised in the **Alignment API format** (`music_alignment.rdf`). The generated alignment contains three high-confidence class correspondences between the developed ontology and PMO/BIBFRAME.

```
1 <map>
2   <Cell>
3     <entity1 rdf:resource=
4       "http://example.org/music-ontology#RecordingSession"/>
5     <entity2 rdf:resource=
6       "http://performedmusicontology.org/ontology/RecordingSession"/>
7     <measure rdf:datatype="xsd:float">0.99</measure>
8     <relation>=</relation>
9   </Cell>
10 </map>
11
12 <map>
13   <Cell>
14     <entity1 rdf:resource=
15       "http://example.org/music-ontology#Event"/>
16     <entity2 rdf:resource=
17       "http://id.loc.gov/ontologies/bibframe/Event"/>
18     <measure rdf:datatype="xsd:float">0.9837</measure>
19     <relation>=</relation>
20   </Cell>
21 </map>
22
23 <map>
24   <Cell>
25     <entity1 rdf:resource=
26       "http://example.org/music-ontology#Concert"/>
27     <entity2 rdf:resource=
28       "http://performedmusicontology.org/ontology/Concert"/>
29     <measure rdf:datatype="xsd:float">0.9818</measure>
30     <relation>=</relation>
31   </Cell>
32 </map>
```

Listing 3: Extract from `music_alignment.rdf`

Table 7: Ontology Alignment Correspondences

Source	Target	Confidence	Relation
music:RecordingSession	pmo:RecordingSession	0.9900	=
music:Event	bibframe:Event	0.9837	=
music:Concert	pmo:Concert	0.9818	=

5.4 Why Only Three Correspondences Were Found

Although the developed ontology contains 56 classes, AML identified only three high-confidence correspondences. This result is expected and reflects differences in ontology scope rather than limitations of the matching process.

The primary reasons are:

- **Different modelling objectives:** The developed ontology focuses on the popular music industry, including artists, albums, tours, awards, venues, record labels, and commercial music production. PMO focuses primarily on musical performances, recording sessions, performed works, and performance-related concepts.
- **Domain-specific classes:** Many classes in the developed ontology, such as `Award`, `Tour`, `RecordLabel`, `MusicAward`, and `MembershipSituation`, do not have direct equivalents in PMO.
- **Conservative matching strategy:** AML returns only high-confidence mappings. Instead of producing numerous uncertain correspondences, the tool identifies only those relationships that are strongly supported by lexical and structural evidence.
- **Limited naming overlap:** Classes such as `Concert` and `RecordingSession` exist in both ontologies with nearly identical names and meanings, resulting in confidence values above 0.98. Most other classes are modelled differently and therefore do not satisfy AML’s matching criteria.
- **Cross-ontology interoperability:** The correspondence between `music:Event` and `bibframe:Event` demonstrates that AML successfully identified semantic equivalence across ontologies even when the target concept originates from a reused external vocabulary.

Therefore, the discovery of only three correspondences should not be interpreted as a failure of the alignment process. Rather, it indicates that only three concepts were sufficiently similar across the ontologies to justify a high-confidence semantic equivalence relationship.

5.5 Strengths and Weaknesses of the Alignment

5.5.1 Strengths

- **High-confidence mappings:** All identified correspondences achieved confidence scores above 0.98, indicating strong semantic similarity between the matched concepts.
- **Interoperability with external standards:** The mappings connect the developed ontology with PMO and BIBFRAME, enabling potential integration with external music and cultural heritage knowledge bases.
- **Standard alignment format:** The Alignment API RDF format ensures compatibility with ontology engineering tools and ontology merging frameworks.
- **Low risk of incorrect mappings:** By focusing on only the strongest correspondences, AML reduces the likelihood of introducing incorrect semantic relationships.

5.5.2 Weaknesses

- **Limited overlap between ontologies:** Only three correspondences were identified because the developed ontology focuses on the popular music industry, whereas PMO focuses on performed music and performance modelling.
- **No property-level alignment:** The alignment primarily identified class correspondences. Property mappings would require additional manual analysis and validation.
- **Different conceptual granularity:** PMO contains specialised performance-related concepts that are outside the scope of the developed ontology, reducing the number of possible correspondences.
- **Potential need for manual refinement:** Additional useful mappings may exist but would require expert review and manual validation beyond automated matching.

6 Step 5: KG-Based RAG Application

6.1 Application Architecture

A **Knowledge Graph-based Retrieval-Augmented Generation (KG-RAG)** [2] application was developed in Python (`rag_llm.py`). The system enables users to ask natural-language questions about the music knowledge graph through a command-line interface (CLI).

The application combines a materialized RDF knowledge graph with a local Large Language Model (LLM) running through **Ollama** [4] using the **Phi-3**[3] model. RDFLib is used for SPARQL-based retrieval, while the LLM generates human-readable answers from the retrieved facts.

The architecture consists of four layers:

1. **Knowledge Graph Loading Layer:** The materialized Turtle file `music_ontology-materialized_new.ttl` is loaded into an RDFLib Graph object at startup.
2. **Intent Detection Layer:** User questions are analyzed using keyword matching to determine the appropriate SPARQL query template.
3. **Knowledge Retrieval Layer:** SPARQL queries are executed against the in-memory knowledge graph to retrieve counts, entities, ontology classes, or ontology properties.
4. **LLM Generation Layer:** Retrieved facts are provided as context to the local Phi-3 language model via Ollama. The model generates a natural-language answer based solely on the retrieved knowledge graph information.

The resulting pipeline is:

Question → SPARQL Retrieval → Knowledge Graph Facts → Phi-3 LLM → Natural Language Answer

6.2 Demonstration

The KG-RAG application was tested using the materialized music knowledge graph. The system successfully retrieves information from the graph and generates human-readable responses through the LLM.

Sample Q&A Session (CLI output)

```
> python rag_llm.py
Music Knowledge Graph RAG + Ollama
Type a question or 'exit'
```

Question: How many albums are there?

Generated Answer:
There are 541 albums in the Music Knowledge Graph.

Question: How many artists are there?

Generated Answer:
The Music Knowledge Graph contains 50 artists.

Question: What classes exist in the ontology?

Generated Answer:
The ontology contains classes including Musician, Album, Track, Concert, Venue, Genre, Award, Tour, RecordLabel, and MusicWork.

6.3 Supported Entity Types and Queries

The current implementation supports aggregation queries and ontology exploration.

Table 8: Supported KG-RAG Queries

Keyword	OWL Class Targeted	KG Instances
artist	music:Musician	50
album	music:Album	541
track	music:Track	8,664
concert	music:Concert	52
venue	music:Venue	15
class	Ontology Classes	56
propert	Ontology Properties	40+

6.4 Retrieval-Augmented Generation Workflow

Unlike the initial rule-based query application, the KG-RAG system follows a Retrieval-Augmented Generation workflow:

1. The user submits a natural-language question.
2. A SPARQL query retrieves relevant information from the knowledge graph.
3. The retrieved facts are inserted into a prompt.
4. The prompt is sent to the local Phi-3 language model through Ollama.
5. The language model generates a natural-language answer grounded in the retrieved facts.

This approach combines the reliability of knowledge graph retrieval with the natural language generation capabilities of modern Large Language Models.

6.5 Dependencies and Setup

The KG-RAG application requires RDFLib, Ollama, and the Ollama Python client.

```
1 rdflib
2 ollama
```

Listing 4: requirements.txt

Installation:

```
1 pip install rdflib
2 pip install ollama
```

Listing 5: Install dependencies

Install the local Phi-3 model:

```
1 ollama pull phi3
```

Listing 6: Install Phi-3 via Ollama

Run the application:

```
1 python rag_llm.py
```

Listing 7: Run KG-RAG application

6.6 Limitations

- **Rule-based intent detection:** User questions are matched using keyword-based `if/elif` rules. While effective for the supported queries, the system may fail to recognize complex or differently phrased questions. Future work could employ semantic parsing or LLM-based intent classification.
- **Limited query coverage:** The current implementation supports a predefined set of entity types and aggregation queries (artists, albums, tracks, concerts, venues, classes, and properties). More advanced competency questions would require additional SPARQL templates and query logic.
- **No multi-hop reasoning:** Questions requiring complex graph traversals, such as identifying artists through multiple relationships, are not currently supported. Such queries would require more sophisticated SPARQL generation and reasoning capabilities.
- **Result size limitation:** Entity-listing queries use `LIMIT 10` to keep command-line output readable. While suitable for demonstration purposes, this restricts access to the full set of matching entities.
- **Local execution environment:** The KG-RAG application operates on a locally loaded RDFLib graph and a locally hosted Phi-3 model through Ollama. While this simplifies deployment and avoids external API dependencies, it may not scale efficiently to very large knowledge graphs or enterprise workloads.

7 Summary and Conclusion

This project successfully demonstrates the complete Semantic Web technology stack applied to the popular music domain. Starting from a relational database, a formal OWL 2 ontology was created with 56 classes and 40+ properties, mapped to relational data via Ontop OBDA, materialised into 73,243 RDF triples, validated with SHACL (0 violations), aligned to an external ontology (PMO/BIBFRAME), and ultimately integrated into a Knowledge Graph-based Retrieval-Augmented Generation (KG-RAG) application.

The resulting system combines Semantic Web technologies with Large Language Models (LLMs), enabling users to query the music knowledge graph using natural language while receiving human-readable responses grounded in factual knowledge graph data.

7.1 Key Design Decisions

- **Genre as a first-class entity:** Modelling **Genre** as an OWL class rather than a string literal enables multi-genre classification, genre-level querying, and ontology reasoning.
- **Dynamic venue and artist typing via OBDA:** The OBDA mappings use `{venue_type}` and `{musician_type}` template variables to dynamically assign OWL subclass types during materialisation without requiring separate mappings for each subtype.
- **Property chain for instrument inference:** The `featuresInstrument` property chain (`hasMember` ◦ `playsInstrument`) allows OWL reasoners to infer instruments used by bands automatically.
- **Typed Situation ODP:** Situation-based modelling (`AwardSituation`, `MembershipSituation`) enables contextual facts to be represented as first-class entities, supporting richer semantic queries.
- **Materialised RDF knowledge graph:** Materialising RDF triples into a Turtle knowledge graph simplifies downstream querying, validation, and integration with Semantic Web tools.
- **Knowledge Graph-based Retrieval-Augmented Generation:** The final application combines SPARQL-based retrieval with LLM-based answer generation. RDFLib retrieves factual information from the knowledge graph, while the local Phi-3 model running through Ollama converts retrieved facts into natural-language responses.
- **Fully local deployment:** The KG-RAG system operates entirely on local resources using RDFLib, Ollama, and Phi-3, eliminating dependence on external AI APIs and improving reproducibility.
- **Null-safe label-to-label mapping:** The award–artist mapping uses `WHERE label_id IS NOT NULL` to prevent null key errors and maintain graph consistency.

Overall, the project demonstrates how ontology engineering, OBDA, RDF knowledge graphs, SHACL validation, ontology alignment, and Retrieval-Augmented Generation can be integrated into a complete end-to-end Semantic Web solution. The resulting system provides both structured semantic knowledge and natural-language interaction capabilities, illustrating the practical value of combining Knowledge Graphs with modern Large Language Models.

7.2 Project Repository

The complete source code, ontology files, OBDA mappings, SHACL validation artifacts, materialized knowledge graph, ontology alignment files, and RAG implementation are available in the project GitHub repository:

https://github.com/Muhammad123255/SAIKS_Project

The repository contains all resources required to reproduce the experiments and results presented in this report.

References

- [1] joebeachcapital. *Spotify Kaggle Dataset*. <https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs>. Accessed: 2026-06-01.
- [2] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [3] Microsoft Research. *Phi-3-mini-4k-instruct Language Model*. Accessed: 2026-06-25. 2024. URL: <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct>.
- [4] Ollama Team. *Ollama: Local Large Language Models*. Accessed: 2026-06-25. 2026. URL: <https://ollama.com/>.